



Vilnius
University

MINI PROJECT

INTERNATIONAL
CYBERSECURITY
AND
CYBERINTELLIGENCE

2025



VILNIUS UNIVERSITY
FACULTY OF MATHEMATICS AND INFORMATICS
INSTITUTE OF COMPUTER SCIENCE
CYBERSECURITY LABORATORY

International Cybersecurity and Cyberintelligence 1st year Research Project

NAS : Next.js Attack Surface

Done by:

Giovanni Menon

Raphael Toubol

Christopher Samson Nyandoro

Supervisor:

prof. dr. Agnė Brilingaitė

Vilnius
2025

Contents

Introduction 4

1 Related Work 5

2 Methodology 6

 2.1 Requirements 6

 2.2 Tool Design and Architecture 6

3 Results 7

4 Discussion 7

Conclusions and Future Work 8

References 9

Introduction

Web Application development and tooling has transformed drastically since the introduction and widespread adoption of single-page applications and full-stack JavaScript frameworks. This modern web application development architecture provides significant advantages in terms of tooling, user and developer experience, however introduce increased complexity in the realm of security.

Necessity for complex security verification comes from the implications of insecure web applications in the modern age. This coupled with the multi-faceted attack surfaces in web applications dictates an essential requirement for heightened security posturing from developers along with adequate tooling for quickly and effectively completing attack-surface analysis.

Due to the high volume and criticality of vulnerabilities in web-app specific code and the plethora of vulnerabilities the javascript third-party dependency libraries this is a focus area of this scientific research.

This report outlines the implementation and analysis of a Golang config file based tool specifically for the purpose of discovering and surfacing vulnerabilities in API Routes and dependencies in React based NextJS full stack applications directly from their source code repositories. From here after the tool will be referred to as NAS (Next-Attack-Surface).

1. Related Work

This section outlines and relates the scientific work of this paper to the foundational academic literature it is built upon. The security of JavaScript front-end frameworks has been scientifically investigated and academically discussed in the paper [4] "Evaluating Security for JavaScript-based Frontend Frameworks". The author discusses, compares and contrasts the security features and inherent vulnerabilities of several popular front-end frameworks including React, Vue & Svelte. The NextJS framework extends react from the client to the server, which is why the underlying security features and vulnerabilities of react are so important.

A considerable and documented vulnerability source in modern web applications comes directly from developers leveraging third-party dependencies. The paper [5] "Thou Shalt Not Depend on Me: Analysing the Use of Outdated JavaScript Libraries on the Web" analyses data from over 133,000 finding 37% to have outdated libraries. With a lead time between vulnerability discovery and patching to be over a year the industry cannot rely on automatically upgrading packages and needs to leverage vulnerability scanning tools.

Node Package Manager (NPM) further compounds this issue as discussed in [3] "On the impact of security vulnerabilities in the npm package dependency network". This vulnerability propagation throughout the JavaScript ecosystem occurs as a result of the widespread, and high availability of outdated and vulnerable libraries. This dependency relationship and vulnerability reality can occur via transitivity when the developer leveraging a package inherits a vulnerability from a third-party dependency several libraries upstream of the implemented library in the web application codebase.

Focusing directly the research discussed in [1] "Managing Security Vulnerabilities Introduced by Dependencies in React.JS JavaScript Framework" targeted mitigation strategies for react specific dependency vulnerabilities are discussed.

In specific cases identification of vulnerabilities requires automated static analysis. These static analysis tools analyse code without executing it, [2] "Study of JavaScript Static Analysis Tools for Vulnerability Detection in Node.js Packages" provides a complete comparison of the capabilities of several tools geared toward the Node.js runtime. This highlights the power of static analysis while also demonstrating some of the shortcomings in the current available solutions.

The research discussed and thoroughly documents the probable security issues in NextJS applications inherited from the wider JavaScript, NPM and React ecosystems. The existence of static analysis tools is also outlined however there is an evident need for easily accessible tooling which consolidates the capabilities of effective vulnerability detection tools into one quick and easy developer focused tool. Consolidated tooling removes the burden on the developer to implement and use a wide variety of tools in order to simply confirm the safety of their application. This research and implementation addresses this need by developing such a tool.

2. Methodology

This section describes in detail the logic and requisites of our implementation, showing the underlying logic and the procedures used to carry out our product. Section ?? is devoted to a description of our product, while Section 2.1 describes the requisites behind our work.

2.1. Requirements

The main goal of this tool was to implement and consolidate attack surface analysis tooling from multiple other tools into one simple and quick interface. To adequately address the needs of developers, and complete a well rounded attack surface analysis there are several key requirements which need to be addressed:

1. Scanning of package dependencies (and dev-dependencies) for vulnerabilities
2. Scanning of code for:
 - Clear-text passwords
 - Exposed CORS attack surface
 - Remote code execution
 - API keys
 - Sensitive data in comments & cleartext
3. associating identified vulnerable packages with CVE numbers
4. integration with proven static analysis tools
5. surfacing the information in a visual format
6. configuration via file

Meeting these key requirements would equate to a tool that could be leveraged by all NextJS developers for dependency scanning, static analysis & identification of information leakage through external api's.

2.2. Tool Design and Architecture

Golang was the choice of language for developing this project due to the third party libraries available (verified vulnerability free), its ease of use, simple syntax and strong typing. Based on the requirements some clear design decisions were made:

Dependency scanning requirements are met by creating a module which leverages established dependency scanning API's. This decision reduces complexity and removes the onus of maintaining an up-to-date database with vulnerability to package relationships. NAS is using the OSV-Scanner API's based on its simple request and response formats and its ability to return vulnerabilities from batch lists of packages from varying environments. NAS parses the packages and sends them in batch form to the API, this decision was made instead of integrating their cli tool directly, this results in the outcomes:

1. Individually querying dependencies and dev dependencies can result in more granular reporting.
2. package-lock.json, package.json, yarn.lock, pnpm.lock & bun.lock can all be parsed depending on the specific package manager the NextJS project is running.

A code scanning module has been developed which uses REGEX to scan for clear-text data leaks, API Keys, exposed CORS and Remote code execution attack surfaces. This package summarizes the used REST methods, searches for "Access-Control-Allow-Origin.*", "Eval()" and "Key, Token, Secret & Password" strings.

Currently the outcomes of the scanner modules are presented to the user in a PDF document outlining the vulnerable packages and their ID's.

3. Results

4. Discussion

Conclusions and Future Work

References

- [1] Terzi Anastasia and Bibi Stamatia. Managing security vulnerabilities introduced by dependencies in react.js javascript framework. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering - Companion (SANER-C)*, pages 126–133, 2024.
- [2] Tiago Brito, Mafalda Ferreira, Miguel Monteiro, Pedro Lopes, Miguel Barros, José Fragoso Santos, and Nuno Santos. Study of javascript static analysis tools for vulnerability detection in node.js packages. *IEEE Transactions on Reliability*, 72(4):1324–1339, 2023.
- [3] Alexandre Decan, Tom Mens, and Eleni Constantinou. On the impact of security vulnerabilities in the npm package dependency network. In *Proceedings of the 15th International Conference on Mining Software Repositories, MSR '18*, page 181–191, New York, NY, USA, 2018. Association for Computing Machinery.
- [4] Edvin Hellquist. Evaluating security for javascript-based frontend frameworks. Master’s thesis, Umeå University, Department of Computing Science, 2025.
- [5] Tobias Lauinger, Abdelberi Chaabane, Sajjad Arshad, William Robertson, Christo Wilson, and Engin Kirda. Thou shalt not depend on me: Analysing the use of outdated javascript libraries on the web. In *Proceedings 2017 Network and Distributed System Security Symposium, NDSS 2017*. Internet Society, 2017.